

■ Relatório de Auditoria de Segurança

Solo Learning — Solen / Collegium App

Data: 28 de August de 2026

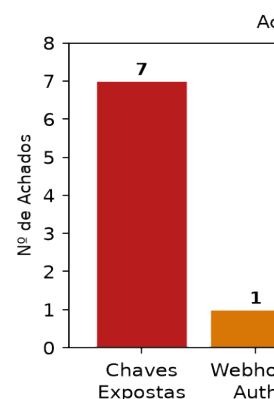
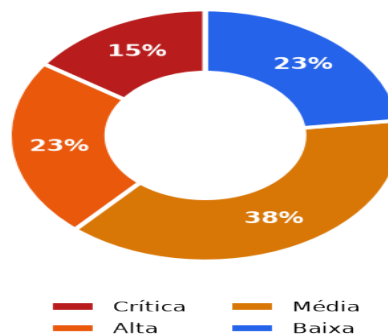
Escopo: Código-fonte completo do repositório (backend + frontend)

Nota Metodológica: A auditoria cobriu as cinco categorias padrão adaptadas à stack detectada: Node.js + Fastify + Prisma ORM + PostgreSQL (backend), Expo/React Native (frontend mobile). O mecanismo de isolamento de tenant é manual via filtro por `request.user.instituicao / institutionId` nas queries Prisma — não há RLS no banco. Todos os achados foram verificados no código-fonte real. Nenhuma especulação foi incluída.

1. Resumo Executivo

Severidade	Qtd
Crítica	2
Alta	3
Média	5
Baixa	3
TOTAL	13

Achados por Severidade



2. Pontos Fortes (Controles Verificados)

■ admin.ts — Isolamento de tenant em todas as rotas

Todas as rotas de /admin aplicam globalmente authenticate + validateTenantStatus + validateInstitution via addHook. As queries filtram explicitamente por request.user.instituicao em todas as operações CRUD.

■ superadmin.ts — Role ADMIN verificado em todos os handlers

Todos os endpoints de /superadmin verificam role === 'ADMIN' via addHook global, com fallback 401/403 correto.

■ professor.ts — Isolamento de dados do professor

Todas as queries filtram por professorId do usuário autenticado via turmaDisciplinas.some.

■ bounty.ts — Verificação de posse nos endpoints críticos

POST /:id/seen e POST /:id/response verificam bug.userId === request.user.id antes de permitir alterações.

■ quests.ts — POST /daily/submit verifica posse da entrega

Verifica delivery.userId !== userId antes de processar, com caminho correto para Raid.

■ wrong-answers retry — Verificação de posse

GET e POST de /wrong-answers/:id verificam wrongAnswer.userId !== userId antes de processar.

■ bcrypt em senhas de professores/arquitetos

Senhas de professores e arquitetos são hashadas com bcrypt (10 rounds) na criação e reset.

■ auth.ts — Middleware centralizado

O plugin de autenticação é registrado globalmente e aplica jwtVerify() em todas as rotas protegidas.

3. Achados Detalhados

CRÍTICA [F1] JWT_SECRET com fallback hardcoded no código-fonte

Categoria: CAT-4 Chaves Expostas

Arquivo:linha — backend/src/plugins/auth.ts:8

Trecho:

```
secret: process.env.JWT_SECRET || 'supersecret_solen_key_123'
```

Descrição: O segredo JWT tem um fallback literal 'supersecret_solen_key_123' diretamente no código-fonte. Qualquer pessoa com acesso ao repositório pode forjar tokens JWT válidos para qualquer usuário, incluindo ADMIN, se o JWT_SECRET não for definido no ambiente de produção.

Impacto: Escalada de privilégio total — forja de tokens arbitrários.

Correção sugerida: Remover o fallback e adicionar validação de startup que lance exceção se JWT_SECRET estiver ausente ou for o valor padrão conhecido.

CRÍTICA [F2] Credenciais reais commitadas no .env rastreado pelo git

Categoria: CAT-4 Chaves Expostas

Arquivo:linha — backend/.env:1-11

Trecho:

```
DATABASE_URL="postgresql://solen_user:solen_password@..."
```

```
JWT_SECRET="supersecretjwtkey"
```

```
GEMINI_API_KEY="AQ.Ab8RN6Lx..."
```

```
NVIDIA_API_KEY="nvapi-ZDg3qfS5..."
```

```
SMTP_PASS="ufer gsek fyhh rmhb"
```

Descrição: O arquivo .env com credenciais reais (banco de dados, JWT, APIs Gemini/Nvidia, senha de app Gmail) está presente no repositório e rastreado pelo git. O commit '25b0923 — Remove chave no .env' (2026-08-03) modificou o arquivo mas ele ainda existe na árvore de trabalho com valores reais. Credenciais antigas permanecem acessíveis no histórico do git.

Impacto: Comprometimento de todas as credenciais: banco, AI, e-mail.

Correção sugerida: 1. Adicionar backend/.env ao .gitignore imediatamente. 2. Rotacionar TODAS as credenciais expostas. 3. Usar git-filter-repo para expurgar o arquivo do histórico.

ALTA [F3] Senha padrão de Arquiteto hardcoded e exposta na UI

Categoria: CAT-4 Chaves Expostas

Arquivo:linha — backend/src/routes/superadmin.ts:99,348 /
frontend/app/(superadmin)/dashboard.tsx:377,716,1179

Trecho:

```
const rawPassword = password || 'Solen2026';
```

Descrição: A senha padrão 'Solen2026' está hardcoded no código de criação e reset de Arquitetos, e também exibida literalmente na UI do superadmin. Qualquer Arquiteto que nunca trocou a senha (isFirstAccess=true) pode ser acessado com essa senha conhecida publicamente pelo código.

Impacto: Comprometimento de contas de Arquiteto com acesso administrativo à instituição.

Correção sugerida: Gerar senhas aleatórias fortes na criação/reset de arquitetos. Remover a senha do texto da UI e comunicá-la por canal seguro.

ALTA [F4] Credenciais PostgreSQL hardcoded no docker-compose.yml

Categoria: CAT-4 Chaves Expostas

Arquivo:linha — backend/docker-compose.yml:6-8

Trecho:

```
POSTGRES_USER: solen_user
```

```
POSTGRES_PASSWORD: solen_password
```

```
POSTGRES_DB: solen_db
```

Descrição: O docker-compose.yml define credenciais padrão previsíveis para o banco PostgreSQL. Esses mesmos valores aparecem no DATABASE_URL do .env. Se o container for exposto ou o compose usado em produção sem override, a senha é trivialmente conhecida.

Impacto: Acesso direto ao banco de dados de produção.

Correção sugerida: Usar variáveis de ambiente (\${POSTGRES_PASSWORD}) sem defaults no docker-compose e definir senhas fortes em .env não rastreado.

ALTA [F5] Endpoint de webhook de billing sem verificação de assinatura

Categoria: CAT-5 Webhook sem Autenticação

Arquivo:linha — backend/src/routes/webhooks.ts:6-43

Trecho:

```
fastify.post('/billing/webhook', async (request, reply) => {  
  // Sem autenticação, sem verificação de assinatura HMAC  
  if (type === 'invoice.paid') { /* ATIVA conta */ }
```

Descrição: O webhook de billing não valida a assinatura do payload (HMAC-SHA256 do Stripe/Asaas). Qualquer ator pode enviar um POST para /webhooks/billing/webhook com 'type': 'invoice.paid' e um institutionId arbitrário, ativando gratuitamente qualquer conta bloqueada ou inadimplente.

Impacto: Fraude: ativação de contas inadimplentes sem pagamento; DoS: bloqueio de contas pagantes.

Correção sugerida: Implementar verificação de assinatura HMAC-SHA256 usando o segredo de webhook do gateway de pagamento antes de processar qualquer evento.

MÉDIA [F6] GET /quests/subject-stats/:userId sem verificação de posse/tenant

Categoria: CAT-3 IDOR

Arquivo:linha — backend/src/routes/quests.ts:2800-2892

Trecho:

```
fastify.get('/subject-stats/:userId', ..., async (request, reply) => {  
  // Verifica role, mas NÃO verifica se userId pertence à instituição do caller  
  const student = await prisma.user.findUnique({ where: { id: userId } });
```

Descrição: O endpoint retorna estatísticas de desempenho de qualquer aluno por userId. Professores ou Arquitetos de uma instituição podem consultar dados de alunos de outras instituições. Não há verificação de que o userId pertence ao tenant do chamador.

Impacto: Exposição de dados de desempenho de alunos de outras instituições.

Correção sugerida: Adicionar verificação: confirmar que o student.institutionId (ou student.instituicao) é igual ao do caller antes de retornar dados.

MÉDIA [F7] GET /bounty/active expõe todos os bugs reports sem isolamento de tenant**Categoria:** CAT-3 IDOR**Arquivo:linha** — backend/src/routes/bounty.ts:192-208**Trecho:**

```
const bugs = await prisma.bountyBug.findMany({
// sem filtro por instituição
orderBy: { createdAt: 'desc' }
});
```

Descrição: O endpoint GET /bounty/active retorna todos os bug reports de todas as instituições para qualquer usuário autenticado. Inclui nome e matrícula do aluno que reportou o bug via o campo include.user.

Impacto: Vazamento de PII (nome e matrícula) de alunos de todas as instituições.

Correção sugerida: Filtrar por instituição do caller: where: { instituicao: request.user.instituicao }. Para ADMIN, manter a visão global sem filtro.

MÉDIA [F8] GET /admin/matrix/audit aceita institutionId arbitrário via query param**Categoria:** CAT-1 Isolamento de Tenant**Arquivo:linha** — backend/src/routes/admin.ts:1040-1058**Trecho:**

```
const { institutionId, instituicao: targetInstName, ... } = request.query;
if (institutionId) {
targetInst = await prisma.institution.findUnique({ where: { id: institutionId } });
}
```

Descrição: O endpoint /admin/matrix/audit aceita um institutionId via query string e usa esse ID sem verificar se ele é o mesmo da instituição do Arquiteto autenticado. Um ARQUITETO pode passar o UUID de qualquer outra instituição e visualizar toda a matriz de auditoria dela (quests, entregas, logs de ação).

Impacto: Cross-tenant data leak de métricas e logs de auditoria.

Correção sugerida: Ignorar institutionId/instituicao da query para ARQUITETO e usar sempre request.user.institutionId. Apenas ADMIN deve poder filtrar por outra instituição.

MÉDIA [F9] Endpoint PUT /quests/:id (editar quest) sem verificar posse do professor**Categoria:** CAT-2 Permissão Definida no Navegador**Arquivo:linha** — backend/src/routes/quests.ts:549-572**Trecho:**

```
fastify.put('/:id', ..., async (request, reply) => {
if (request.user.role !== 'PROFESSOR' && request.user.role !== 'ADMIN')
return 403;
// Quest atualizada SEM verificar se o professor criou ou tem vínculo com ela
const updated = await prisma.quest.update({ where: { id } ... });
```

Descrição: A rota PUT /quests/:id verifica apenas o papel (PROFESSOR/ADMIN), mas não verifica se o professor tem vínculo com a disciplina/turma da quest. Um professor de qualquer instituição pode editar o enunciado de qualquer quest de qualquer outra instituição se souber o UUID.

Impacto: Sabotagem de quests de outras instituições; cross-tenant data tampering.

Correção sugerida: Verificar se a quest pertence a uma turma cujo professor leciona: adicionar where: { turmaAlvo: { turmaDisciplinas: { some: { professorId } } } }.

MÉDIA [F10] POST /quests/mock-boss sem verificar se professor pertence à turma

Categoria: CAT-2 Permissão Definida no Navegador

Arquivo:linha — backend/src/routes/quests.ts:574-673

Trecho:

```
fastify.post('/mock-boss', ..., async (request, reply) => {  
  // Verifica role PROFESSOR|ADMIN, mas:  
  const turma = await prisma.turma.findUnique({ where: { id: turmaId } });  
  // Sem verificar se o professor está vinculado a essa turma
```

Descrição: O endpoint que cria um BOSS para uma turma verifica apenas se o usuário é PROFESSOR ou ADMIN, mas não verifica se o professor tem vínculo (TurmaDisciplina) com a turma especificada. Um professor pode invocar um BOSS em qualquer turma, inclusive de outras instituições.

Impacto: Disruption de aulas: professor malicioso pode interromper aulas de outra turma/instituição.

Correção sugerida: Adicionar verificação: confirmar que o professor possui um TurmaDisciplina ativo para o turmaId recebido.

BAIXA [F11] Senha placeholder insegura armazenada em hash para alunos sem login

Categoria: CAT-4 Chaves Expostas

Arquivo:linha — backend/src/routes/admin.ts:550, 638, 680

Trecho:

```
password: 'INITIAL_SUMMONING_CODE_LOGIN' // não é bcrypt hash!  
password: 'SUMMONING_CODE'  
password: 'RESET_TO_SUMMONING_CODE'
```

Descrição: Alunos criados via bulk ou individual recebem senhas em plaintext não-hasheadas no banco de dados. O login usa bcrypt.compare, então essas contas não conseguem fazer login diretamente, mas a senha em plaintext viola as políticas de armazenamento seguro e pode confundir sobre o estado da conta.

Impacto: Baixo risco de exploração direta; violação de boas práticas de armazenamento.

Correção sugerida: Usar await bcrypt.hash('SUMMONING_CODE', 10) mesmo para placeholders, ou marcar isFirstAccess=true sem definir senha.

BAIXA [F12] CORS wildcard (*) habilitado no servidor

Categoria: CAT-4 Chaves Expostas

Arquivo:linha — backend/src/server.ts:23-27

Trecho:

```
server.register(cors, { origin: '*', ... });
```

Descrição: O CORS está configurado com origin: '*', permitindo que qualquer domínio faça requisições cross-origin para a API. Em produção, isso abre a API para ser chamada de qualquer site malicioso, potencializando CSRF/phishing.

Impacto: Informativo/baixo — JWT mitiga o risco principal, mas é má prática.

Correção sugerida: Restringir origin a domínios específicos (app mobile não usa CORS, mas o frontend web usa). Definir lista allowlist de domínios.

BAIXA

[F13] Token JWT passado como query parameter (?token=)**Categoria:** CAT-4 Chaves Expostas**Arquivo:linha** — backend/src/plugins/auth.ts:13-15 / frontend/services/api.ts:322**Trecho:**

```
const queryToken = (request.query as any)?.token;  
if (queryToken && !request.headers.authorization) {  
  request.headers.authorization = `Bearer ${queryToken}`;  
}
```

Descrição: O plugin de auth suporta JWT via query parameter, e o frontend o usa para download de templates Excel. Tokens em URLs são armazenados em logs de servidor, histórico do browser e referrer headers, expondo-os desnecessariamente.

Impacto: Vazamento de token em logs/referrer headers.

Correção sugerida: Para downloads de arquivo autenticados, usar cookies httpOnly ou gerar URLs pré-assinadas de curta duração em vez de passar o token na URL.

4. Recomendações Priorizadas

P1 — Imediato

- F2: Revogar todas as credenciais do .env commitado (Gemini, Nvidia, Gmail, DB, JWT). Adicionar ao .gitignore. Expurgar do histórico git.
- F1: Remover fallback do JWT_SECRET. Adicionar validação de startup.
- F5: Implementar verificação de assinatura HMAC no webhook de billing.

P2 — Curto Prazo

- F3: Gerar senha aleatória forte para Arquitetos. Remover "Solen2026" da UI e do código.
- F4: Parametrizar credenciais do docker-compose via variáveis de ambiente.
- F9: Verificar posse do professor ao editar quests.
- F10: Verificar vínculo professor-turma no endpoint mock-boss.

P3 — Médio Prazo

- F6: Verificar tenant do aluno em /subject-stats/:userId.
- F7: Filtrar bug reports por instituição do caller em /bounty/active.
- F8: Restringir institutionId da query de audit à instituição do Arquiteto.

P4 — Melhorias

- F11: Hashar senhas placeholder de alunos.
- F12: Restringir CORS origin para lista de domínios permitidos.
- F13: Substituir JWT em query param por URLs pré-assinadas de curta duração.

5. Issues para o GitHub

--- ISSUE 1 ---

Título: [Segurança] JWT_SECRET hardcoded e credenciais reais no repositório

Labels: security, severity:critical

Achados cobertos: F1, F2

Descrição:

Descrição

Duas falhas críticas relacionadas à gestão de segredos foram identificadas:

1. JWT_SECRET com fallback hardcoded (`auth.ts:8`)

O plugin de autenticação usa `supersecret\_solen\_key\_123` como fallback quando `JWT\_SECRET` não está definido. Qualquer pessoa com acesso ao repositório pode forjar tokens JWT válidos para qualquer usuário, incluindo ADMIN.

2. Credenciais reais commitadas no `.env` (`backend/.env`)

O arquivo `.env` com `DATABASE\_URL`, `JWT\_SECRET`, `GEMINI\_API\_KEY`, `NVIDIA\_API\_KEY` e `SMTP\_PASS` está rastreado pelo git. Mesmo após o commit 'Remove chave no .env' (25b0923), o arquivo existe na árvore de trabalho e versões antigas estão no histórico.

Evidência:

- `backend/src/plugins/auth.ts:8`: `secret: process.env.JWT\_SECRET || 'supersecret\_solen\_key\_123'`
- `backend/.env:1-11`: DATABASE_URL, JWT_SECRET, GEMINI_API_KEY, NVIDIA_API_KEY, SMTP_PASS

Impacto:

- Forja de tokens JWT → escalada de privilégio total
- Acesso ao banco de dados de produção
- Uso indevido das cotas de API (Gemini/Nvidia)
- Acesso à caixa de e-mail institucional

Sugestão de Correção:

1. ****IMEDIATO****: Revogar todas as credenciais expostas (rotacionar DB password, JWT_SECRET, API keys, app password Gmail)
2. Adicionar `backend/.env` ao `.gitignore`
3. Usar `git-filter-repo` para expurgar o `.env` do histórico
4. Remover o fallback do JWT_SECRET em `auth.ts:8`
5. Adicionar validação de startup que rejeite JWT_SECRET ausente ou igual ao padrão conhecido

Critérios de Aceite:

- [] Todas as credenciais rotacionadas e confirmadas inválidas
- [] `backend/.env` adicionado ao `.gitignore` e não aparece em `git status`
- [] `auth.ts:8` sem fallback literal; servidor falha ao iniciar sem JWT_SECRET válido
- [] `git log --all -- backend/.env` retorna apenas commits de remoção

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Senha padrão "Solen2026" hardcoded e credenciais do Docker expostas

Labels: security, severity:high

Achados cobertos: F3, F4

Descrição:

Descrição

1. Senha padrão de Arquiteto (`superadmin.ts:99`)

A senha 'Solen2026' está hardcoded na criação e reset de Arquitetos, e também exibida na UI do superadmin (`dashboard.tsx:377,716,1179`). Qualquer Arquiteto que não trocou a senha pode ser comprometido.

2. Credenciais PostgreSQL no docker-compose.yml (`docker-compose.yml:6-8`)

As credenciais do banco (user/password/db) estão hardcoded com valores previsíveis que coincidem com os do DATABASE_URL.

Evidência:

- `backend/src/routes/superadmin.ts:99`: `const rawPassword = password || 'Solen2026';`
- `backend/docker-compose.yml:6-8`: `POSTGRES\_PASSWORD: solen\_password`

Impacto:

- Comprometimento de todas as contas de Arquiteto não alteradas
- Acesso direto ao banco de dados se o container for exposto

Sugestão de Correção:

1. Gerar senhas aleatórias fortes (ex: `crypto.randomBytes(16).toString('hex')`) na criação/reset de arquitetos
2. Remover 'Solen2026' da UI — comunicar a senha por canal seguro
3. Usar variáveis de ambiente sem defaults no docker-compose: `\${POSTGRES\_PASSWORD:?}`

Crítérios de Aceite:

- [] `superadmin.ts` não contém strings literais de senha
- [] `dashboard.tsx` não exibe a senha padrão na UI
- [] `docker-compose.yml` usa `\${POSTGRES\_PASSWORD}` sem valor default
- [] Arquitetos com `isFirstAccess=true` recebem senha gerada aleatoriamente

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Webhook de billing sem verificação de assinatura HMAC

Labels: security, severity:high

Achados cobertos: F5

Descrição:

Descrição

O endpoint `POST /webhooks/billing/webhook` não valida a assinatura do payload recebido. Qualquer ator pode enviar um POST forjado com `type: 'invoice.paid'` e um `institutionId` arbitrário para ativar gratuitamente qualquer conta bloqueada ou inadimplente. O inverso também é possível: cancelar contas pagantes.

Evidência:

- `backend/src/routes/webhooks.ts:6-43`: sem validação de assinatura

Impacto:

- Fraude: ativação gratuita de contas
- DoS: bloqueio de contas pagantes

Sugestão de Correção:

Implementar verificação de assinatura HMAC-SHA256 usando o segredo de webhook do gateway (ex: Stripe: `stripe.webhooks.constructEvent(body, sig, secret)`, Asaas: verificar header `asaas-access-token`)

Critérios de Aceite:

- [] Webhook rejeita payloads sem assinatura válida com HTTP 400
- [] Segredo de webhook armazenado em variável de ambiente
- [] Teste de rejeição de payload forjado documentado

--- FIM ISSUE 3 ---**--- ISSUE 4 ---**

Título: [Segurança] IDOR e cross-tenant em endpoints de dados de alunos e audit

Labels: security, severity:medium

Achados cobertos: F6, F7, F8

Descrição:**Descrição**

Três endpoints expõem dados de outros tenants:

F6 — GET /quests/subject-stats/:userId (`quests.ts:2800`)

Retorna desempenho de qualquer aluno sem verificar se pertence à instituição do caller.

F7 — GET /bounty/active (`bounty.ts:192`)

Retorna todos os bug reports (incluindo nome e matrícula) de todas as instituições.

F8 — GET /admin/matrix/audit (`admin.ts:1040`)

Aceita `institutionId` via query param sem verificar se pertence ao caller.

Evidência:

- `quests.ts:2800`: sem verificação de tenant do student
- `bounty.ts:192`: `prisma.bountyBug.findMany({})` sem filtro
- `admin.ts:1040-1058`: aceita institutionId arbitrário

Impacto:

- Vazamento de PII de alunos (nome, matrícula)
- Cross-tenant data leak de métricas pedagógicas e logs de auditoria

Sugestão de Correção:

F6: Verificar `student.institutionId === request.user.institutionId`

F7: Adicionar `where: { instituicao: request.user.instituicao }` (exceto ADMIN)

F8: Ignorar `institutionId` da query para ARQUITETO; usar `request.user.institutionId`

Critérios de Aceite:

- [] `/subject-stats/:userId` retorna 403 para aluno de outra instituição
- [] `/bounty/active` retorna apenas bugs da instituição do caller
- [] `/admin/matrix/audit` ignora institutionId arbitrário para ARQUITETO
- [] Testes automatizados cobrindo os três cenários

--- FIM ISSUE 4 ---**--- ISSUE 5 ---**

Título: [Segurança] Falta de verificação de posse em edição de quests e invocação de boss

Labels: security, severity:medium

Achados cobertos: F9, F10

Descrição:**Descrição****F9 — PUT /quests/:id (quests.ts:549)**

Professor pode editar o enunciado de qualquer quest sem verificar vínculo com a turma.

F10 — POST /quests/mock-boss (quests.ts:574)

Professor pode invocar um BOSS em qualquer turma (inclusive de outras instituições) sem verificar TurmaDisciplina.

Evidência:

- `quests.ts:549-572`: `prisma.quest.update({ where: { id } })` sem verificação de posse
- `quests.ts:574-673`: `prisma.turma.findUnique({ where: { id: turmaId } })` sem verificação de vínculo

Impacto:

- Sabotagem de quests de outras instituições
- Disrupção de aulas: BOSS invocado em turmas não gerenciadas pelo professor

Sugestão de Correção:

F9: `prisma.quest.update({ where: { id, turmaAlvo: { turmaDisciplinas: { some: { professorId } } } })`

F10: Verificar `prisma.turmaDisciplina.findFirst({ where: { turmaId, professorId } })` antes de prosseguir

Crítérios de Aceite:

- [] Professor não consegue editar quest de turma sem vínculo
- [] Professor não consegue invocar boss em turma sem vínculo
- [] Ambos retornam 403 em caso de acesso indevido

--- FIM ISSUE 5 ---**--- ISSUE 6 ---**

Título: [Segurança] Melhorias de boas práticas: senhas placeholder, CORS e JWT em URL

Labels: security, severity:low

Achados cobertos: F11, F12, F13

Descrição:**Descrição****F11 — Senhas placeholder em plaintext (admin.ts:550,638,680)**

Alunos criados via bulk recebem strings plaintext como senha no banco.

F12 — CORS wildcard (server.ts:24)

CORS aberto para qualquer origem (`origin: '\*'`).

F13 — JWT em query parameter (auth.ts:13-15, api.ts:322)

Token JWT exposto em URLs de download, logs e referrer headers.

Evidência:

- `admin.ts:550`: `password: 'INITIAL\_SUMMONING\_CODE\_LOGIN'`
- `server.ts:24`: `origin: '\*'`
- `api.ts:322`: `?token=\${encodeURIComponent(token)}`

Impacto:

Violação de boas práticas; risco baixo mas acumulado.

Sugestão de Correção:

F11: Usar `bcrypt.hash('SUMMONING\_CODE', 10)` para senhas placeholder

F12: Definir lista de domínios permitidos no CORS

F13: Gerar URL temporária pré-assinada via endpoint separado para downloads

Crítérios de Aceite:

- [] Senhas de alunos armazenadas como hash bcrypt
- [] CORS restrito a domínios da lista allowlist
- [] Downloads de template não expõem JWT na URL

--- FIM ISSUE 6 ---
